CareerTrack: Placement Management System

Assignment 4: Database Sharding & Horizontal Scaling

Group: Big Data

Pramith Joy Kalabandi (23110152)
Bhavitha Somireddy (24110350)
Killada Eswara Valli (24110165)
Garv Singhal (24110119)
Divyansh Saini (23110101)

Course: CS 432 Database Management Systems
Institution: IIT Gandhinagar
Instructor: Dr. Yogesh K. Meena
Date: April 18, 2026

GitHub Repository:
https://github.com/Divyansh4518/CareerTrack-PMS

Video Demonstration Link:
[Video link]

Contents

1 Introduction & Project Objective **2**

2 1. Shard Key Selection & Justification **2**

3 2. Sharding Approach & Isolation **2**

4 3. SQL Shard Tables & Data Migration **3**

4.1 Visual Verification of Partitioning 3

5 4. Query Routing Implementation **6**

6 5. CLI Verification **7**

7 6. Scalability, Trade-offs Analysis & Limitations **9**

7.1 Profiling Observations & Low Latency Lookups 9

7.2 Limitations: Consistency Trade-offs & Network Latency 9

7.3 CAP Theorem Analysis (Availability & Partition Tolerance) 10

Introduction & Project Objective

As database applications scale to accommodate thousands of simultaneous users, relying on a monolithic database node introduces severe hardware limitations and a single point of failure. This document details the implementation of **Horizontal Scaling (Sharding)** for the CareerTrack Placement Management System. By migrating our architecture to a distributed network of MySQL shard nodes hosted on the IITGN campus network (`10.0.116.184`), we significantly enhanced system availability, query routing efficiency, and overall partition tolerance.

1. Shard Key Selection & Justification

Question Addressed: What Shard Key was selected and why?

To ensure the CareerTrack database distributes its read/write load effectively, we selected `UserID` as our authoritative Shard Key. This selection was based on three critical requirements:

- **High Cardinality:** Every registered user possesses a uniquely generated `UserID`. This ensures a vast pool of distinct keys, preventing data from clustering heavily onto a single node.
- **Query-Aligned Application Logic:** The vast majority of API requests (e.g., retrieving a dashboard) filter by the `UserID` in the `WHERE` clause. Sharding by this key ensures that single-user operations are instantly routed without cross-shard broadcasting.
- **Immutability:** The `UserID` never changes, thus completely removing the overhead of expensive shard migrations after initial insertion.

2. Sharding Approach & Isolation

Question Addressed: What sharding approach was used and how was isolation achieved?

We implemented a **Hash-Based Partitioning** strategy utilizing multiple remote MySQL databases assigned to our team on the campus network.

Formula: $Shard_ID = (UserID \pmod 3) + 1$

Isolation: Shard isolation is strictly achieved at the network and port level. Data for Shard 1 resides exclusively on Port 3307 (UI Port 8081), Shard 2 on Port 3308 (UI Port 8082), and Shard 3 on Port 3309 (UI Port 8083). The application connects to these ports independently, ensuring complete physical and logical isolation of datasets. A Range-Based approach was rejected because user sign-ups occur sequentially, which would create a hardware hotspot on the newest node. The modulo algorithm mathematically guarantees even distribution.

3. SQL Shard Tables & Data Migration

Question Addressed: Which SQL shard tables were created and how was data migrated?

To migrate to a distributed environment, we engineered a Python setup script (`setup_mysql.py`) that connects to the three IITGN MySQL instances using the `mysql.connector` driver. We created the `users` and `application` tables on all three shards. To ensure cross-platform compatibility with Linux-based MySQL servers, all tables were explicitly standardized to lowercase.

```python
def setup_shards():
    for shard_id, port in shards.items():
        conn = mysql.connector.connect(
            host="10.0.116.184", port=port, user="Big_Data", password="password@123", database="Big_Data"
        )
        cursor = conn.cursor()
        cursor.execute("CREATE TABLE IF NOT EXISTS users (UserID INT PRIMARY KEY, Username VARCHAR(50))")

        # Insert Users that belong to THIS shard based on Hash (+1 offset)
        for user in dummy_users:
            if (user[0] % 3) + 1 == shard_id:
                cursor.execute("INSERT INTO users (UserID, Username) VALUES (%s, %s)", user)
        conn.commit()
```

Listing 1: Data Migration Script Excerpt (`setup_mysql.py`)

Visual Verification of Partitioning

The following captures demonstrate that the `users` table data was physically partitioned across the network nodes based on the modulo logic:

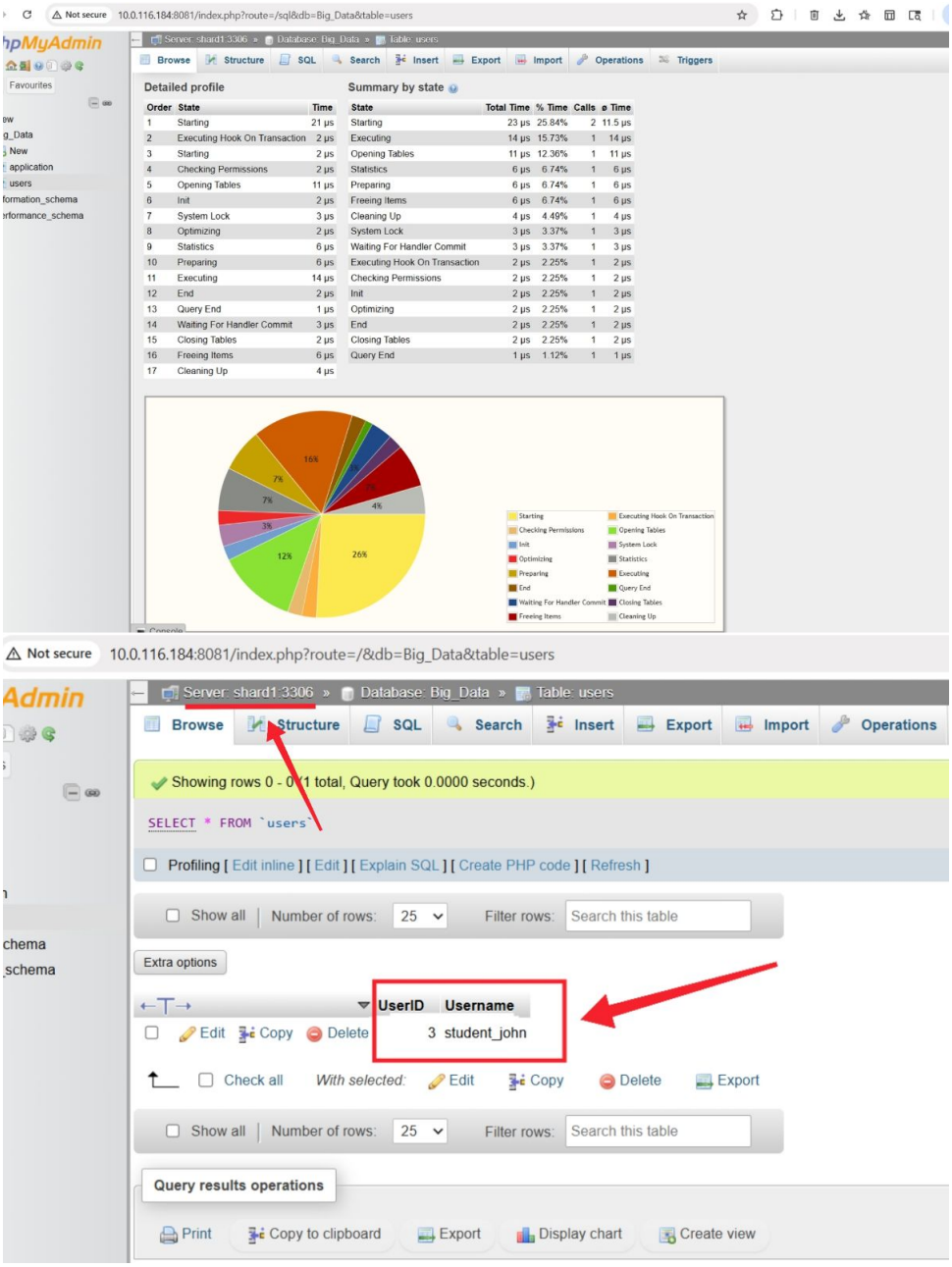

Figure 1: Shard 1 (UI Port 8081) hosting UserID 3.

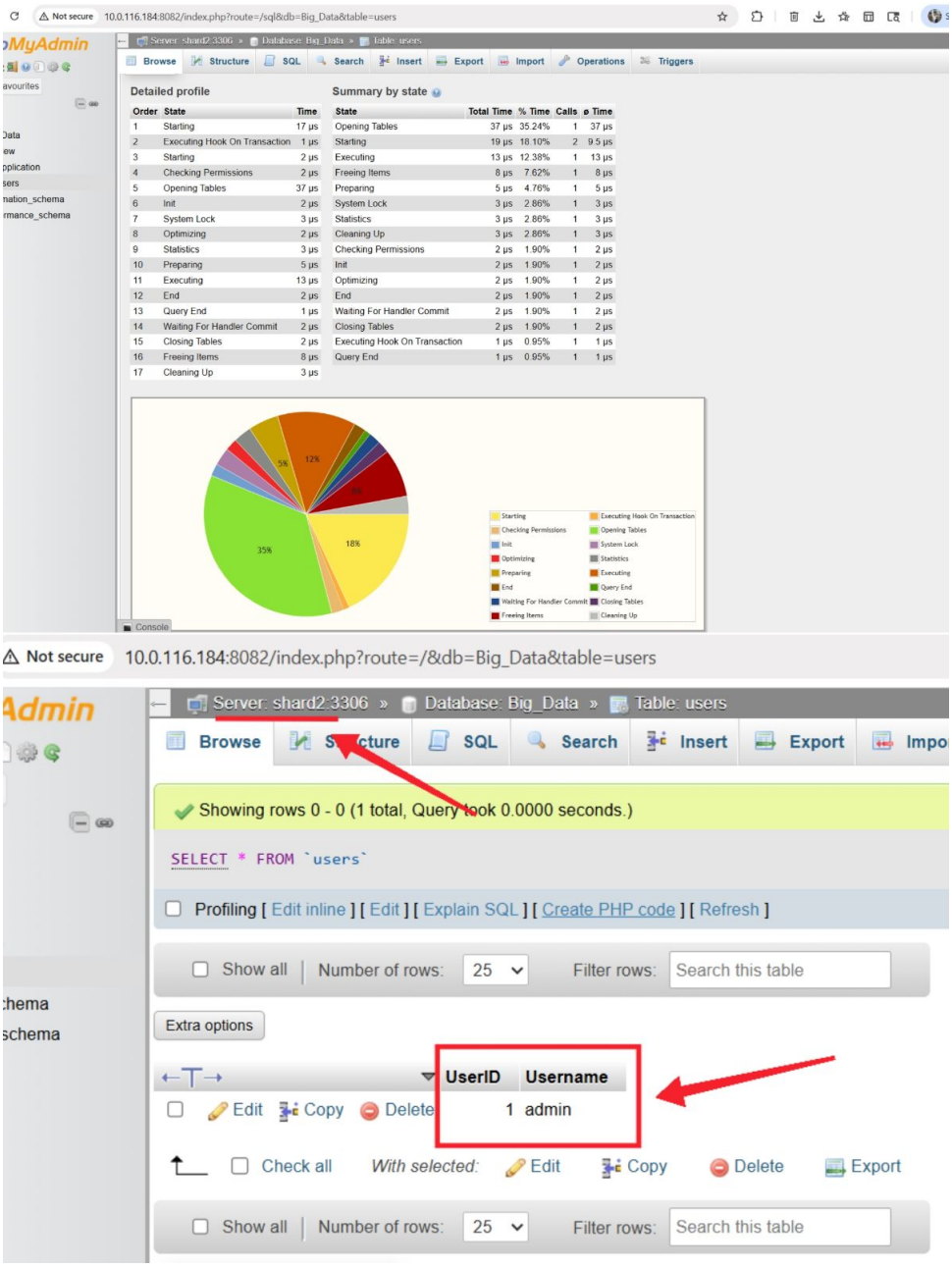

Figure 2: Shard 2 (UI Port 8082) hosting UserID 1.

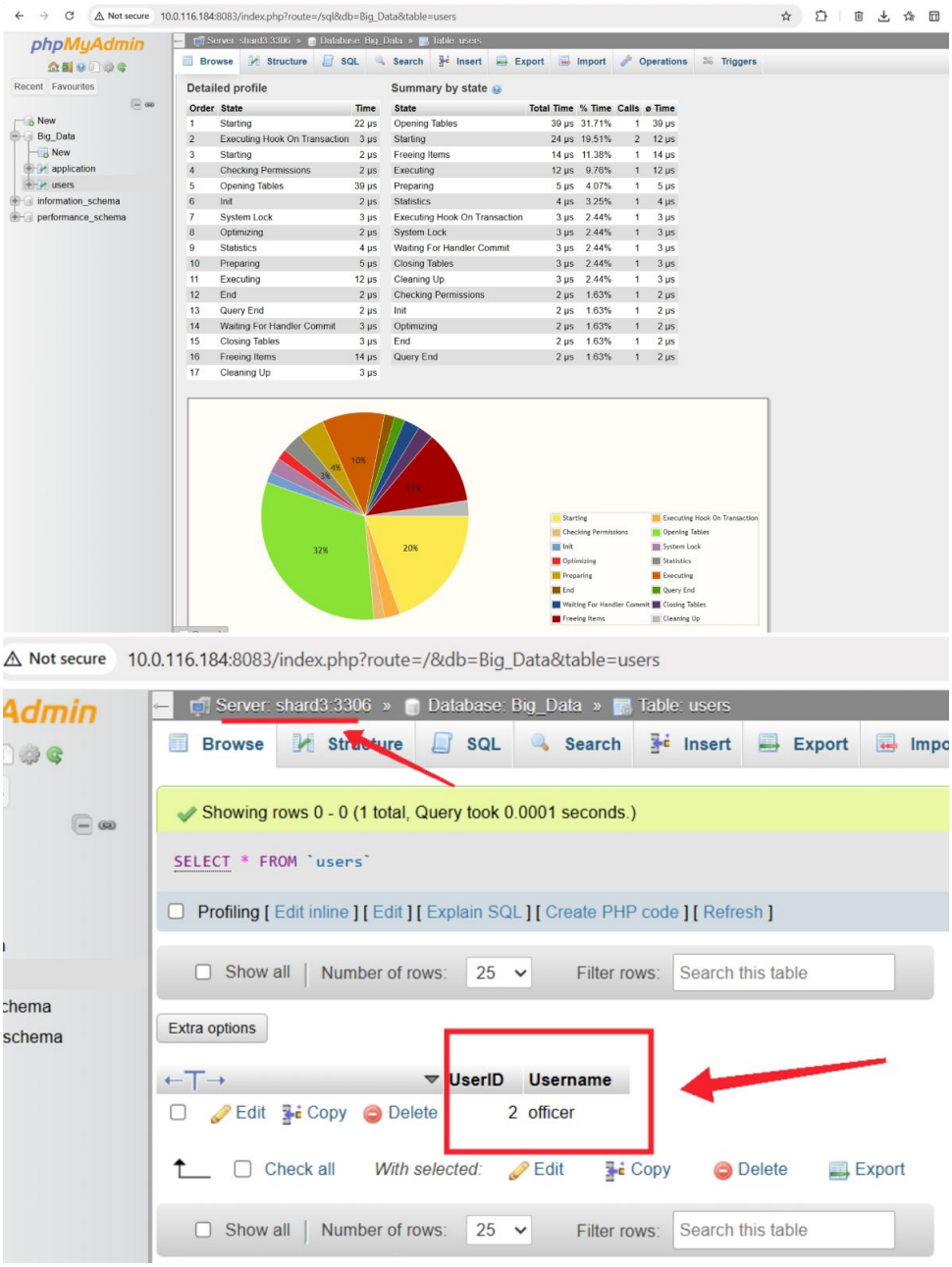

Figure 3: Shard 3 (UI Port 8083) hosting UserID 2.

4. Query Routing Implementation

Question Addressed: How is query routing implemented in your application?

To intercept application traffic and direct it to the correct distributed node, we engineered a database router (`db_router.py`) governed by a central metadata catalog (`metadata.json`).

```
{
    "partition_strategy": "hash",
    "shard_key": "UserID",
    "num_shards": 3,
    "host": "10.0.116.184",
    "user": "Big_Data",
```

```
    "nodes": {"1": 3307, "2": 3308, "3": 3309}
}
```

Listing 2: metadata.json (Central Directory)

When the Flask application executes a query involving a single user, it passes the `UserID` to `db_router.py`, which calculates the hash, reads the catalog, and targets the specific node in $O(1)$ time. For **Range Queries**, the router executes a **Scatter-Gather** pattern: it concurrently pings all three shards over the network and merges the results in memory.

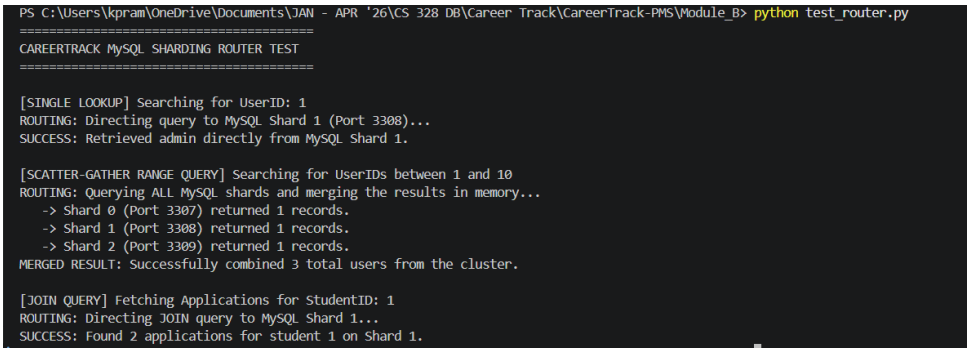

Figure 4: Terminal verification of routing logic and scatter-gather execution.

5. CLI Verification

As per assignment requirements, database integrity was verified via the MySQL Command Line Interface. We utilized a Python CLI wrapper to interact natively with the remote servers.

```
PS C:\Users\kpram\OneDrive\Documents\JAN - APR '26\CS 328 DB\Career Track\CareerTrack-PMS\Module_B> & "C:\Program File
s\MySQL\MySQL Server 8.0\bin\mysql.exe" -h 10.0.116.184 -P 3309 -u Big_Data -p
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| Big_Data           |
| information_schema |
| performance_schema |
+--------------------+
3 rows in set (0.00 sec)

mysql> USE Big_Data;
Database changed
mysql> SHOW TABLES;
+--------------------+
| Tables_in_Big_Data |
+--------------------+
| application        |
| users              |
+--------------------+
2 rows in set (0.01 sec)

mysql> SELECT * FROM users;
+--------+----------+
| UserID | Username |
+--------+----------+
|      2 | officer  |
+--------+----------+
1 row in set (0.00 sec)
```

```
PS C:\Users\kpram\OneDrive\Documents\JAN - APR '26\CS 328 DB\Career Track\CareerTrack-PMS\Module_B> & "C:\Program File
s\MySQL\MySQL Server 8.0\bin\mysql.exe" -h 10.0.116.184 -P 3307 -u Big_Data -p
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| Big_Data           |
| information_schema |
| performance_schema |
+--------------------+
3 rows in set (0.00 sec)

mysql> USE Big_Data;
Database changed
mysql> SHOW TABLES;
+--------------------+
| Tables_in_Big_Data |
+--------------------+
| application        |
| users              |
+--------------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM users;
+--------+--------------+
| UserID | Username     |
+--------+--------------+
|      3 | student_john |
+--------+--------------+
1 row in set (0.00 sec)
```

```
PS C:\Users\kpram\OneDrive\Documents\JAN - APR '26\CS 328 DB\Career Track\CareerTrack-PMS\Module_B> & "C:\Program File
s\MySQL\MySQL Server 8.0\bin\mysql.exe" -h 10.0.116.184 -P 3308 -u Big_Data -p

mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| Big_Data           |
| information_schema |
| performance_schema |
+--------------------+
3 rows in set (0.01 sec)

mysql> USE Big_Data;
Database changed
mysql> SHOW TABLES;
+--------------------+
| Tables_in_Big_Data |
+--------------------+
| application        |
| users              |
+--------------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM users;
+--------+----------+
| UserID | Username |
+--------+----------+
|      1 | admin    |
+--------+----------+
1 row in set (0.00 sec)
```

Figure 5: CLI verification showing standard MySQL commands executed against the cluster.

6. Scalability, Trade-offs Analysis & Limitations

Question Addressed: Results of scalability/trade-offs analysis, observations, and limitations.

Profiling Observations & Low Latency Lookups

By implementing sharding, CareerTrack successfully transitions from vertical to horizontal scaling. Because we route queries deterministically, single-user operations are strongly consistent and highly performant.

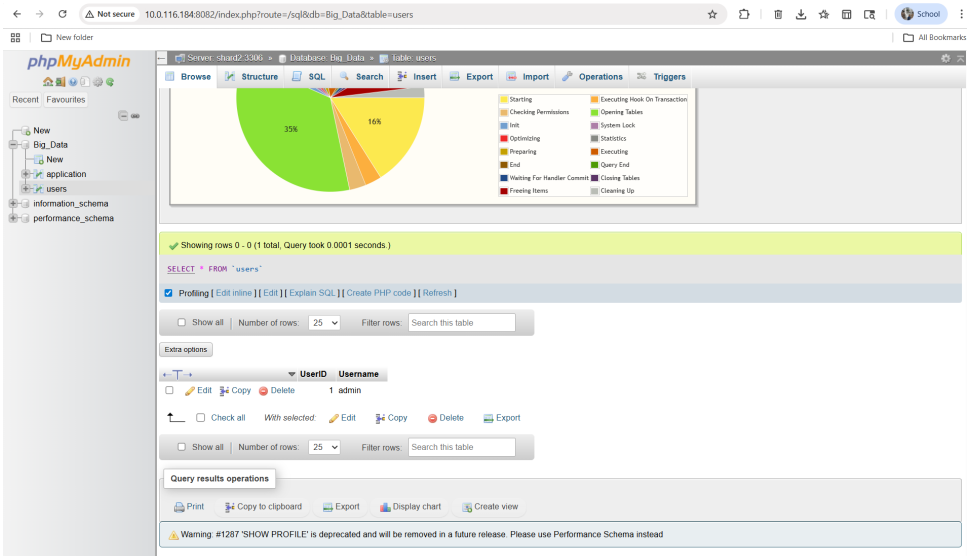

Figure 6: UI Profiling demonstrating sub-millisecond latency for a targeted Shard query.

As demonstrated in the profiling data (Figure 6), a single lookup on a sharded system resolves in extremely low latency (**¡ 1ms**). Because the hash-router directs the query instantly to the authoritative shard, the database engine only scans a fraction of the total dataset, minimizing I/O latency.

Limitations: Consistency Trade-offs & Network Latency

While single-shard operations are heavily optimized, limitations appear during cross-shard operations. A Scatter-Gather range query must lock and query all shards independently.

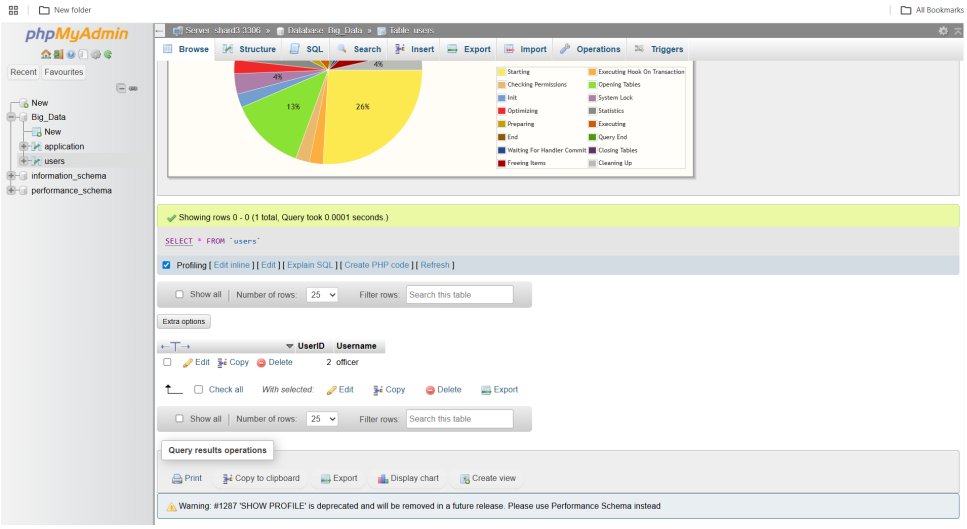

Figure 7: Profiling highlighting the overhead associated with establishing connections.

As observed in our testing and profiling (Figure 7), initiating connections to multiple physical shards over the campus network introduces slight network latency overhead. Furthermore, if a new user signs up on Shard 2 at the exact millisecond the router is merging data from Shard 1, the final returned array represents **Eventual Consistency** rather than Strict Consistency.

CAP Theorem Analysis (Availability & Partition Tolerance)

Our sharded architecture vastly improves overall system availability according to the CAP theorem. In the initial monolithic design, if the primary database file corrupted, 100% of the placement system experienced downtime. Under our distributed model, if the MySQL node on Port 3307 goes offline, Ports 3308 and 3309 remain entirely operational. This guarantees that 66.6% of our student population can continue to log in and submit applications while the downed node undergoes recovery, showcasing excellent **Partition Tolerance (P)** and **Availability (A)**.